

Clase 10

ESTRUCTURAS DE DATOS RECURSIVAS

CC1002 Introducción a la
Programación

Listas

- Vimos que los **structs** sirven para representar información compuesta
 - Son útiles cuando sabemos cuántos datos queremos combinar
- ¿Número indeterminado de elementos?
 - Solución: una lista, que puede tener un largo arbitrario, esto es, contiene una cantidad finita pero indeterminada de elementos

CC1002 Introducción a la
Programación

Listas

- La definición del tipo de dato `lista` es **recursiva**
 - Puede ser vacía (no contiene elementos)
 - Usaremos el valor `listaVacía`
 - Puede ser un dato compuesto con:
 - Un atributo `valor` de cualquier tipo (la **cabeza**)
 - Un atributo `siguiente` de tipo `lista`, que contiene al resto de la lista (la **cola**)

CC1002 Introducción a la
Programación

Listas

- Definición de la estructura para listas

```
import estructura

# lista: valor (any = cualquier tipo) siguiente (lista)
estructura.crear('lista', 'valor siguiente')

# ejemplo con una lista de Strings
unaLista = lista('pedro', lista('juan', lista('diego', listaVacía)))
```

CC1002 Introducción a la
Programación

Listas

```
# Modulo lista.py
# identificador para listas vacias
listaVacia = None

# cabeza: lista -> any
# devuelve la cabeza de una lista (un valor)
def cabeza(unalista):
    return unaLista.valor

# cola: lista -> lista
# devuelve la cola de una lista (una lista)
def cola(unalista):
    return unaLista.siguiente

# vacia: lista -> bool
# devuelve True si la lista esta vacia
def vacia(unalista):
    return unaLista == listaVacia
```

CC1002 Introducción a la
Programación

Listas

```
# Tests

testLista = lista(1, lista(2, lista(3, listaVacia)))

assert cabeza(testLista) == 1
assert cabeza(cola(testLista)) == 2
assert cabeza(cola(cola(testLista))) == 3
assert cola(cola(testLista)) == lista(3, listaVacia)

assert vacia(listaVacia)
assert not vacia(testLista)
assert vacia(cola(cola(cola(testLista))))
```

CC1002 Introducción a la
Programación

Listas

```
>>> # IMPORTANTE: ESTAS LISTAS DEFINIDAS USANDO UN DATO COMPUESTO NO SON MUTABLES
>>> from lista import *
>>> testLista = lista(1, lista(2, lista(3, listaVacía)))
>>> testLista.valor = 4

Traceback (most recent call last):
  File "<pyshell#4>", line 1, in <module>
    testLista.valor = 4
AttributeError: can't set attribute
```

CC1002 Introducción a la
Programación

Listas

```
>>> # Agregar un valor al inicio de la lista
>>> from lista import *
>>> unaLista = lista(1, lista(2, lista(3, listaVacía)))
>>> unaLista = lista(4, unaLista)
>>> # unaLista == lista(4, lista(1, lista(2, lista(3, listaVacía))))
```

CC1002 Introducción a la
Programación

Listas

- Ejemplo: lista con inventario de productos en una juguetería

```
>>> from lista import *
>>> inventario = lista('soldadito', lista('pelota', lista('oso', listaVacía)))
```

- ¿Función para saber si hay pelotas en el inventario?
 - A priori no sabemos el largo del inventario

CC1002 Introducción a la
Programación

Listas

- Funciones para procesar listas:
 - La lista está vacía, o
 - la lista tiene cabeza y cola

```
# hayPelotas: lista(str) -> bool
# determinar si el string 'pelota' esta en la lista unaLista
# ejemplo: hayPelotas(inventario) devuelve True
def hayPelotas(unaLista):
    if vacía(unaLista):
        return False
    else:
        if cabeza(unaLista) == 'pelota':
            return True
        else:
            return hayPelotas(cola(unaLista))

#Test
assert hayPelotas(inventario)
```

CC1002 Introducción a la
Programación

Listas

- Plantilla de función para procesar listas:

```
# procesarLista: lista(any) -> any
# descripcion de la funcion
# ejemplo de uso
def procesarLista(unaLista):
    if vacia(unaLista):
        ...
    else:
        ... cabeza(unaLista) ...
        ... procesarLista(cola(unaLista)) ...

# Tests
...
```

CC1002 Introducción a la
Programación

Listas

- Función para saber cuántas pelotas hay en el inventario
 - A priori no sabemos el largo del inventario
 - Podría haber 0 pelotas en el inventario
 - Podría haber 1 o más pelotas en el inventario
 - ¿Caso base? ¿Expresión para el caso recursivo?

CC1002 Introducción a la
Programación

Listas

```
inventario1 = lista('soldadito', lista('pelota', lista('oso', listaVacía)))
inventario2 = lista('pelota', lista('oso', lista('pelota', lista('robot', listaVacía))))

# cuantasPelotas: lista(str) -> int
# devuelve el numero de veces que el string 'pelota' esta en la lista unaLista
# Ejemplos:
# cuantasPelotas(inventario1) devuelve 1
# cuantasPelotas(inventario2) devuelve 2
def cuantasPelotas(unalista):
    if vacía(unalista):
        return 0
    else:
        if cabeza(unalista) == 'pelota':
            return 1 + cuantasPelotas(cola(unalista))
        else:
            return cuantasPelotas(cola(unalista))

# Tests
assert cuantasPelotas(inventario1) == 1
assert cuantasPelotas(inventario2) == 2
```

CC1002 Introducción a la
Programación

Para la próxima clase

- Leer los Capítulos 9.4, 9.5 y 9.6 del apunte

CC1002 Introducción a la
Programación